PAD-aula10 - Abstract Factory | Prof. Marlon

DISCIPLINA: PADRÕES DE PROJETO | CATEGORIA: GOF (CRIACIONAL) | LINGUAGEM: JAVA

Objetivos da Aula

- Compreender o padrão de projeto Abstract Factory e sua finalidade na criação de famílias de objetos relacionados.
- Identificar cenários onde a acoplagem rígida na criação de objetos impede a escalabilidade do sistema.
- Implementar a abstração de fábricas para garantir independência entre a lógica de negócio e as implementações concretas de componentes.
- Analisar as vantagens de encapsular famílias de objetos, reduzindo a complexidade de manutenção.

Motivação e Contextualização

Em sistemas complexos, frequentemente precisamos criar objetos que pertencem a uma mesma família ou conjunto. Quando utilizamos construtores diretos (new Class()), criamos uma dependência direta (acoplamento) entre o código cliente e a classe concreta. Se o sistema precisar suportar, por exemplo, múltiplos temas visuais (Windows vs. Mac) ou diferentes bancos de dados, o código espalhado pelo sistema tornaria a manutenção um pesadelo.
O padrão Abstract Factory surge como a solução para delegar a responsabilidade de criação para uma interface fábrica, garantindo que o cliente sempre receba objetos compatíveis dentro de uma mesma família.

Exemplo Problemático (Sem o Padrão)

Considere uma aplicação que cria botões e checkboxes. Sem o padrão, o cliente instanciaria as classes diretamente:

```
// Sem o padrão, o cliente conhece as classes concretas
if (sistema == "Windows") {
    Botao b = new BotaoWindows();
    Checkbox c = new CheckboxWindows();
} else {
    Botao b = new BotaoMac();
    Checkbox c = new CheckboxMac();
}
```

Por que isso é um problema? O código cliente está "cheio" de verificações condicionais e conhece os detalhes de implementação das famílias de SO. Adicionar um novo sistema operacional exigiria modificar todos os locais onde objetos dessa família são criados, violando o princípio Aberto/Fechado (Open/Closed Principle).

Solução com o Padrão Abstract Factory

Definimos uma interface `FabricaGUI` que declara métodos para criar cada objeto, e implementamos fábricas concretas para cada família.

```
interface FabricaGUI {
    Botao criarBotao();
    Checkbox criarCheckbox();
}

class FabricaWindows implements FabricaGUI {
    public Botao criarBotao() { return new BotaoWindows(); }
    public Checkbox criarCheckbox() { return new CheckboxWindows()
}

// O código cliente agora trabalha com a interface
FabricaGUI fabrica = new FabricaWindows(); // ou FabricaMac
Botao b = fabrica.criarBotao();
```

Análise da Solução: A criação agora é encapsulada. O cliente não conhece as classes concretas (BotaoWindows, etc.), apenas a interface. Isso permite trocar toda a família de objetos apenas trocando a instância da fábrica no início da aplicação.

Sem o Padrão x Com o Padrão

Sem o Padrão	Com o Padrão
Alto acoplamento	Desacoplado
Difícil de escalar	Fácil expansão de famílias
Código rígido/sujo	Código limpo e modular

Vantagens e Desvantagens

Vantagens: Garante consistência entre produtos, promove o desacoplamento e facilita a troca de famílias de produtos.
Desvantagens: Pode tornar o código mais complexo com muitas interfaces e classes, e é difícil adicionar novos tipos de produtos à interface original sem alterar todas as fábricas.

Exercício 1: Implementação Direta

Implemente um sistema de "Temas de E-commerce" (DarkTheme e LightTheme) contendo componentes `Botao` e `Input` seguindo a estrutura de Abstract Factory.

Exercício 2: Estudo de Caso

Uma empresa de jogos precisa gerar terrenos (Montanha, Deserto) e climas (Chuvoso, Ensolarado). Como aplicar o Abstract Factory para garantir que o clima seja sempre compatível com o tipo de terreno instanciado?

Será que compreendemos o padrão? Então responda:

1. Qual é a diferença fundamental entre Factory Method e Abstract Factory?
2. Por que o Abstract Factory fere o princípio de extensão se precisarmos adicionar um novo componente à interface?
3. Onde devemos instanciar a fábrica concreta na aplicação?

Fechamento Sintetizado

- Abstract Factory é um padrão criacional focado em **famílias de objetos**.
- Promove a abstração entre a lógica de negócio e a criação de instâncias.
- Ideal para sistemas onde a interoperabilidade entre componentes (de uma mesma família) é mandatória.
- A chave para o sucesso é o uso consciente das interfaces de fábrica.